

Q1. HYPERPARAMETER CHOICES

Refer to CONFIG in Section 2 and `build_training_args()` in Section 10 of the notebook.

(a) For each of the following values, state the value you chose and give one concrete reason why that value is appropriate for this task and model: `batch_size`, `learning_rate`, `epochs`, `lora_r`, `lora_alpha`, `max_length`.

For `batch_size` I set the value to 16. The suggested range was 8-32 and I felt that 16 was a happy median. Smaller batches give noisier gradients and risk underfitting, while larger batches can be more stable but lead to overfitting.

For `learning_rate` I set this value to $2e-4$. I initially started with $1e-4$ and felt that there we were underfitting slightly and increasing to $2e-4$ increased performance across metrics without overfitting.

`lora_r` was set to 16. I wanted to see if this maximum suggested value would truly overfit the data, and it didn't, so I kept it across runs and felt it to be a good fit. This higher value may have worked out well given the smaller training dataset size.

`lora_alpha` was set to 32. Here I was simply following the convention of $\text{lora_alpha} = 2 * \text{lora_r}$

(b) The effective LoRA scaling factor is $\text{lora_alpha} / \text{lora_r}$, not `lora_alpha` alone. Given your chosen values, what is your scaling factor? What would change — numerically — if you doubled `lora_r` while keeping `lora_alpha` fixed?

Given the chosen values of `lora_alpha` and `lora_r`, the scaling factor was 2. If I were to double `lora_r` while keeping `lora_alpha` fixed I would have had a scaling factor of 1.

Q2. BASELINE AND THE VALUE OF FINE-TUNING

Refer to the run summary panel in WandB (Overview tab).

(a) Report your `baseline_accuracy` and `baseline_f1_weighted` from the run summary panel. Given that Banking77 has 77 evenly distributed intent classes, what would you expect from a random classifier? How does the untrained ELECTRA baseline compare to that, and what does the difference tell you about the quality of ELECTRA's pre-trained representations before any fine-tuning?

The `baseline_accuracy` for ELECTRA was 0.0075 and `baseline_f1_weighted` for ELECTRA was 0.0009. Given that Banking77 has 77 evenly distributed intent classes, I would have expected a random classifier to have a baseline accuracy of $1/77 = 0.013$ and baseline weighed f1 also = $1/77 = 0.013$. This tells me that a baseline ELECTRA is not well-trained to handle banking specific classification given that it underperforms the baseline accuracy and weighted f1 of a random classifier.

(b) Report your final `val_accuracy` and `val_f1_weighted`. Calculate the `accuracy_gain` over the baseline. In one sentence, explain what this gain demonstrates about the effect of LoRA

fine-tuning.

The final `val_accuracy` was 0.8991 and `val_f1_weighted` was 0.8990. The accuracy gain versus baseline was +0.8916. This gain demonstrates the efficiency of LoRA fine-tuning; just a small amount of parameter updates can have a big effect, especially with an ELECTRA model.

(c) Report your `val_test_gap` (`val_accuracy` minus `test_accuracy`). Is the gap within the 0.02 threshold flagged in the script? What does the size of this gap tell you about whether the model generalises beyond the validation split it was trained against?

The `val_test_gap` was +0.0026. This is well within the 0.02 threshold flagged in the script. This gap size tells us the model generalizes well beyond the validation split it was trained against given that the test accuracy was close and even slightly better.

Q3. TRAINING DYNAMICS

Refer to the WandB Charts tab for all sub-questions.

(a) Describe the shape of your train/loss curve across all training steps. Did loss decrease consistently, plateau, or rise at any point? Identify the approximate step at which the curve flattened and explain what that implies.

The shape of the train/loss curve across all training steps was smooth and began to plateau slightly towards the middle of training, however still decreased as training went on. The approximate step the curve flattened was at 2k. This implies that the model had learned the general intent boundaries and hit a point of diminishing returns. The remaining steps (roughly 6k) were dedicated to resolving the harder, more ambiguous cases (intent pairs that are genuinely similar, like 'cancel order' vs. 'cancel transfer').

(b) Describe the shape of your train/learning_rate curve. Identify the warm-up phase and the decay phase. Explain why starting from zero and ramping up — rather than beginning at the full learning_rate immediately — reduces the risk of destabilizing the pre-trained weights.

The shape of the train/learning rate curve shows a sharp, but linear increase at the early training stages and then an even shallower linear decrease steadily approaching zero, as training advances. What we don't want to do is lose a lot of information that the pretrained weights have based on a small amount of training data early on. We want the learning rate to steadily increase during the warmup phase. The specific risk with pretrained encoders is catastrophic interference. Large early updates can corrupt the rich representational structure ELECTRA built during pretraining, which is exactly the information you're trying to leverage. Warmup preserves that structure during the most fragile phase of fine-tuning.

(c) Examine your train/grad_norm curve. `max_grad_norm = 1.0` is set in `build_training_args()`. Describe what you observe: does `grad_norm` stay consistently below 1.0, spike above it, or hit the ceiling repeatedly? What does a `grad_norm` that repeatedly hits 1.0 indicate about training stability, and what would you adjust in `CONFIG` to address it?

The grad norm is spikey, reaching values around 5.0. This indicates that it is consistently and significantly exceeding the `max_grad_norm = 1.0` clip threshold throughout training. Every step

where the norm hits 5.0, the gradients are being rescaled down by a factor of 5 before the update is applied. This tells me that the raw gradient signal is large and volatile. The optimizer wants to take large steps but the clip is doing its job and intervening every time to keep updates bounded. The first lever I would lower is `lora_r`. Reducing from 16 to 8 for example gives the adapters less capacity, which could smooth the gradient landscape. Additionally lowering the learning rate (while increasing epochs), and/or increasing warmup steps could help as well.

Q4. PEFT EFFICIENCY

Refer to the parameter count printed at runtime and the LoRA setup in Section 9.

(a) Report the exact number of trainable parameters and the trainable_param_pct from your run summary. The LoRA adapter adds approximately $2 \times \text{lora_r} \times d$ parameters per adapted layer, where d is the hidden dimension of the query, key, or value projection. Using the printed counts, verify that this formula is consistent with your output.

The exact number of trainable parameters was 1,534,541 and the trainable parameter percent was 1.382%. This formula is consistent with our output.

(b) Name two specific deployment advantages of storing LoRA adapter weights separately from the frozen base model, compared to deploying a fully fine-tuned model.

1. Storage efficiency

The base model is stored once and shared across all tasks. Each LoRA adapter is only a few MB, versus a full 110M parameter copy per task with full fine-tuning.

2. Hot-swappable inference

The base model loads into GPU memory once. Different adapters can be swapped on top per request without reloading the base, making multi-task serving significantly cheaper than maintaining separate fully fine-tuned models.

Q5. WANDB EXPERIMENT COMPARISON

Run the script a second time with one CONFIG value changed — suggested: `lora_r` halved or `learning_rate` doubled. Then open both runs in the WandB dashboard before answering.

⚠ Complete additional runs before answering Q5

All three sub-questions require side-by-side data from two or more runs. Answers based on a single run cannot be assessed.

(a) Using the Runs table, report the val_f1_weighted and trainable_param_pct for multiple runs side by side. Which run produced the better outcome, and by how much?

The first run had lower learning rate than the second run. The second run is the run I have been reporting on for this assignment. LR first run = $1e-5$ and LR second run = $2e-5$. The second run (higher LR) produced a better outcome. The final test accuracy of the higher LR model was 0.89

(compared to 0.85) and the final test weighted f1 for the higher LR model was 0.89 (compared to 0.85).

(b) Examine train/grad_norm for both runs. Did the change you made to CONFIG affect gradient stability? Describe what you observe and explain the connection between the changed parameter and the grad_norm profile.

Examining the grad norm curves for both runs the profiles appear largely similar. They have comparable spike magnitudes and roughly the same overall stability pattern. Doubling the learning rate did not produce a meaningful difference in gradient stability.

Grad norm measures the magnitude of raw gradients before the learning rate is applied. The learning rate scales the weight update but does not directly influence the gradient computation itself. So, it is the expected outcome that changing the learning rate alone leaves the grad norm profile largely unchanged as the loss landscape the model is traversing is the same in both runs, producing similar gradient magnitudes regardless of how large a step the optimizer takes in response.

Q6. ERROR ANALYSIS — CONFUSION PAIRS

Refer to the first table printed in Section 12 of the notebook.

The confusion pairs table lists cases where the model predicted the wrong intent. Each row shows a true intent (what the query actually meant), the intent the model predicted instead, and the number of times that specific mismatch occurred. A pair is considered confused when the model consistently mistakes one intent for the other.

(a) From your table, identify the three pairs with the highest mismatch counts. For each pair, quote the true intent and the predicted intent by name, then explain in one or two sentences what specific wording, topic, or customer goal the two intents share that you think caused the model to treat them as the same.

Based on the logs, the three highest mismatch pairs (all tied at count = 4) are:

1. declined_card_payment → predicted as card_not_working Both intents describe a card failing at the point of use. From the customer's perspective "my card was declined" and "my card isn't working" are nearly synonymous phrases for the same frustrating experience, giving the model very little linguistic signal to distinguish them.

2. direct_debit_payment_not_recognised → predicted as card_payment_not_recognised Both intents share the core customer goal of disputing an unrecognised charge, and the phrase "payment not recognised" appears in both intent names. The only distinguishing signal is the payment method (direct debit vs card), which may be absent or ambiguous in how customers phrase their queries.

3. transfer_not_received_by_recipient → predicted as pending_transfer Both intents concern a transfer that hasn't arrived yet. A customer asking, "my transfer hasn't been received" and one asking about a "pending transfer" are describing the same observable situation from

slightly different angles, and the model conflates them because the surface level topic is identical while the distinction lies in subtle intent: one is reporting a problem, the other is a status enquiry.

(b) Propose one concrete intervention — either a data change or a model change — that would specifically reduce your most frequent confusion pair. Explain the mechanism by which your proposed change would help.

Adding more training examples where the wording makes the difference between the two intents obvious could help performance. For `declined_card_payment` use examples that mention a specific purchase being refused at a store or online. For `card_not_working` use examples where the card itself is physically broken or not being recognized by a device.

Right now, the model confuses these two because the training examples for both intents sound too similar. Adding examples with clearer, more distinct wording gives the model better signal for where to draw the line between them.

Q7. ERROR ANALYSIS — PRECISION AND RECALL

Refer to the second table printed in Section 12 of the notebook.

(a) Identify the weakest intent by F1 from your output. Report its precision, recall, and support (number of validation examples). State clearly whether the weakness is driven by low precision, low recall, or both.

`card_not_working` is the weakest intent with an F1 of 0.625. Precision is 0.526, recall is 0.769, and support is 13 validation examples. The weakness is primarily driven by low precision. Recall is reasonably strong at 0.769.

(b) Low precision means the model predicts this intent too often — it fires on queries that belong to other classes. Low recall means the model misses true instances of this intent — assigning them to other classes instead. Given which applies to your weakest intent, describe the specific type of error the model is making in concrete operational terms: what does a customer experience when this error occurs?

Low precision means the model is predicting `card_not_working` too liberally. It fires on queries that belong to other intents. In concrete operational terms, a customer contacts support about a declined payment or an unrecognized transaction and the system incorrectly routes them to a `card_not_working` resolution flow. That customer then gets directed toward card replacement or technical troubleshooting steps when what they needed was help disputing a charge or investigating a declined transaction. They have been sent down the wrong support path entirely, which is a frustrating and time-wasting experience.

(c) The confusion pairs table and the per-class precision/recall table both describe model errors, but they expose different failure modes. Give one example of a failure that would be visible

The confusion pairs table shows the specific intents getting mixed up with each other. It shows the direction of the error, meaning which class is being mistaken for which. The per class

precision and recall table shows the severity of the problem at the level of each individual intent but without showing where the misclassified examples are actually going.

A failure visible in the per class table but not the confusion pairs table would be a case where `card_not_working` misclassifications are spread thinly across many different intents rather than concentrated on one. The low precision shows up clearly in the per class table, but because no single target class accumulates enough errors to surface as a top confused pair, it never appears in the confusion table at all.

Q8. BUSINESS SUMMARY

Write 2-3 paragraphs addressed to a non-technical customer service manager. Your summary must cover all four of the following points:

- State what the model does and what Banking77 is.*
- Quantify the accuracy improvement over the untrained baseline in plain language (no technical jargon — express it as a percentage-point gain or a ratio).*
- Reference the val → test gap and explain in plain terms what it means for real-world reliability.*
- Name your weakest intent from Q7 and describe the operational consequence for the customer service team — specifically, what happens to a customer whose query falls into that category.*

The fine-tuned ELECTRA model is designed to classify different customer service requests into many different representative categories based on the language of the request. The Banking77 dataset is what we used to fine-tune the model. It contains over 13,000 different customer service queries that are pre-classified into 77 fine grained intent categories covering things like card issues, transfers, account management, payments, and identity verification. This is the data that the ELECTRA model was fine-tuned on.

The accuracy of the fine-tuned ELECTRA model had an increase of just over 89% compared to the untrained baseline model. We also saw that the fine-tuned ELECTRA model performed just as well on the test data set compared to the validation data set which demonstrates that the model performs well on unseen data (i.e. expected to do well (89% accuracy) on new/incoming customer queries).

The intent or customer service request type that the model has the most difficulty classifying are 'card not working' requests. The model has low precision when predicting this class, meaning it is likely to misclassify customer queries as 'card not working' issues. For example if a customer contacts support about a declined payment or an unrecognized transaction and the system incorrectly interprets this to be a 'card not working' issue.